TP 1

Implémentation de l'algorithme Value Iteration pour résoudre un MDP

Consignes :

1. **Lire attentivement l'énoncé du TP dans son intégralité** : Il est important de bien comprendre toutes les parties de l'énoncé avant de commencer l'implémentation. Prenez le temps de lire attentivement chaque section pour éviter toute confusion.
2. **Lire et comprendre le code avant de le compléter ou l'exécuter** : Cette consigne vise à souligner l'importance de bien comprendre l'ensemble du sujet avant de réaliser le travail demandé. Une bonne préparation permet de mieux structurer votre approche et d'éviter des erreurs pendant l'implémentation.
3. **Avant de poser une question, vérifier si la réponse n'est pas déjà présente dans l'énoncé** : je répondrai volontiers à vos questions si la réponse n'est pas déjà dans l'énoncé.

Objectifs

Ce TP vise à mieux comprendre et maitriser les modèles décisionnels de Markov et le fonctionnement de l'algorithme de Value Iteration (VI).

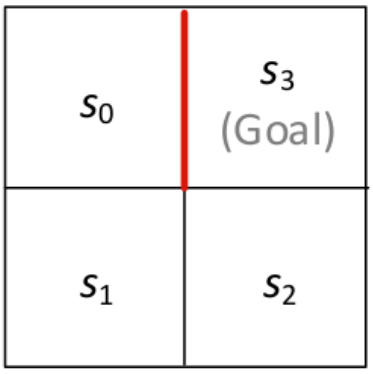
Présentation des éléments

La méthode de l'**itération de la valeur** (Value Iteration) est une technique de programmation dynamique permettant de trouver la fonction de valeur optimale en résolvant de manière itérative l'équation de Bellman. Vous devez implémenter cet algorithme afin de résoudre un problème simple.

Dans le dossier python sur icampus, vous avez :

- le fichier Grid4Env.py qui représente l'environnement Gille à 4 états (voir image ci-dessous)
- Le fichier valueIteration.py incomplet
- Le fichier main-VI.py qui permet de lancer l'ensemble

Figure 1. Grille 4 états.



Partie 1 : Value Iteration

Pour calculer la politique optimale, vous devez implémenter l'algorithme Value Iteration afin de calculer la fonction de valeur en complétant le fichier (valueIteration.py).

Sachant que selon le code fourni :

- Les actions possibles sont ("left","right","up","down").
- Le déplacement dans la version déterministe est sûr à 100% alors votre fonction de transition est **déterministe**. Par exemple, faire l'action "down" depuis l'état S0 amène le robot à l'état S1 avec une probabilité de 1.
- Le robot ne peut pas sortir de la grille ; les actions qui amènent en dehors de la grille n'ont pas d'effet (le robot reste sur place). Par exemple, faire l'action "left" depuis l'état S0 amène le robot à l'état S0 avec une probabilité de 1.
- Les séparations rouges indiquent un mur et donc ont l'effet de bordure de la grille (si tentative de les traverser, l'action n'a pas d'effet). Par exemple, faire l'action "right" depuis l'état S0 amène le robot à l'état S0 avec une probabilité de 1.
- L'objectif est d'atteindre l'état S3. L'action immédiate qui permettra le robot d'atteindre cet objectif lui donnera 10 points (récompense = 10). Dans ce problème, c'est l'action "up" depuis S2. Toute autre action dans son parcours est pénalisée d'un point (récompense = -1).

Voici le pseudo-code de l'algorithme value iteration.

```
Value Iteration algorithm
    Initialize delta to 1
    Initialize the value function to 0 for all states
    Set iteration counter to 1
    While delta is higher than epsilon, repeat :
        Reinitialize delta to 0
        For all states :
            V_s_old = copy of value function for this state
            For all actions
                Calculate V(s,a) and keep V(s) with max value
            Set V(s) = max V for all actions
            delta = max( [delta, abs(V_New[s]-v_s_old)] )
        print v(s)
        iteration counter +1
```

Dans le fichier valueIteration.py :

1.1. Implémentez l'algorithme de Value Iteration qui prend en paramètre les éléments de votre MDP en plus de gamma = 0.99 et epsilon = 0.1.

1.2. Afficher les valeurs des 4 états à la fin de chaque itération.

1.3. Implémentez la fonction **policy** qui rend l'action à réaliser selon l'état passé en paramètre d'entrée.

1.4. Implémentez la fonction **playEpisode** qui prend comme paramètre d'entrée un état de démarrage et itère en appliquant la meilleure action jusqu'à atteindre l'état final (S3). Afficher les informations (s,a,s,r) à chaque itération. Tester la fonction avec l'état de démarrage S0.

La fonction **playEpisode doit respecter les étapes suivantes :**

- tant que l'état terminal n'est pas atteint
    - chercher la meilleure action à partir de l'état actuel (avec policy pi)
    - récupérer l'état suivante en appliquant cette action (avec T) : si T est probabiliste (à corriger au moment de passer à la question 2), il faut choisir aléatoirement parmi les états destinataires possibles
    - récupérer la récompense à la suite de l'application de cette action
    - imprimer les informations (s,a,s',r)
    - avancer dans l'environnement en mettant à jour l'état actuel

1.5. Prenez le temps de bien commenter votre code.

Partie 2 : Transition probabiliste

2.1. Dans un nouveau fichier Grid4ProgEnv.py, créer une version probabiliste de votre problème grille à 4 cases. Proposer une fonction de transition probabiliste.

2.2. Modifier le code de PlayEpisode pour prendre en compte les transitions probabilistes.

2.3. Testez Value Iteration sur le modèle probabiliste. Que notez-vous sur le nombre d'itérations nécessaire pour converger vers une politique optimale ?

Partie 3 : Gamma

2.1. Modifier la valeur de gamma à 0.05 et retester votre code.

2.2. Comparer le nombre d'itérations nécessaires pour converger à une fonction de valeur optimale pour le cas de gamma = 0.99 et gamma = 0.05. Expliquer ce changement ?

Partie 4 : Grille 6

3.1. Proposer le fichier gameGrid6.py avec un modèle MDP qui représente le labyrinthe présenté dans la FIGURE 2. Considérer une fonction de transition déterministe. Le robot a une récompense de 10 points s'il arrive à sortir du plan depuis S4 (action "down" à partir de S4) et une récompense de 100 s'il arrive à sortir à partir de S1 (action "up" à partir de S1), toute autre action est pénalisée de 1 point.

3.2. Combien d'itérations VI a besoin pour converger avec gamma = 0.99. Comparer avec le nombre d'itérations nécessaire pour la grille 2x2.

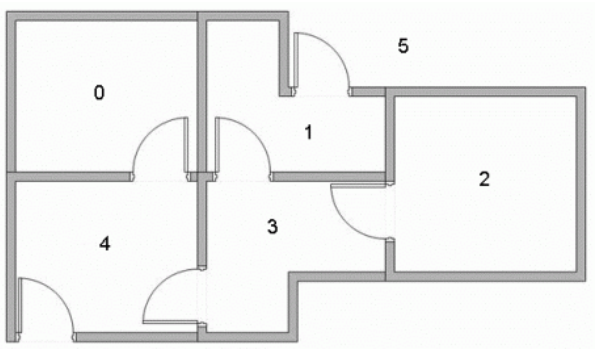


FIGURE 2. Labyrinthe 6 états